FRONT-END DEVELOPMENT

ASSIGNMENT 3 TEMPLATE

STUDENT NAME: Hariharan S

YEAR: II

COLLEGE CODE:1455

COLLEGE NAME: Tagore College of Arts and Science

DEPARTMENT: Computer Science

SEMESTER: IV

ROLL NO: unm1455222407723

NMID: 614D89761591EB584744FC4BCEE6FE5F

DATE: 10/03/2026

PROJECT TITLE: JavaScript Interactivity for Event Registration

---

1. Problem Statement:

Small event organizers need a way to ensure the data they collect is accurate and to see registrations update instantly without refreshing the page. While Assignment-2 focused on look and feel, this assignment adds functionality to validate user input and dynamically manage participant lists.

---

2. Objective:

To implement JavaScript for client-side validation and DOM manipulation. The goal is to ensure only correct data is submitted and to provide immediate visual feedback by updating the "Registered Students" table automatically.

3.Requirements:

This section details the functional requirements implemented to move the project from a static design to an interactive web application:

1. **Form Validation:**

- **Mandatory Checks**: Wrote conditional logic in script.js to verify that Name, Date of Birth, and College fields are not empty.
- **Format Verification**: Used string methods to check for the presence of the "@" symbol in the Email field and ensured the Phone number contains exactly 10 digits.
- **Selection Logic**: Added checks to confirm that an event has been chosen from the dropdown and that both Gender and Mode radio buttons have a "checked" status.

3.2 Confirmation Checkbox Validation:

- **Prerequisite Check**: Implemented an if statement to check the .checked property of the declaration checkbox.
- **Error Prevention**: If the checkbox is unselected, the script triggers a warning alert and halts the submission process to ensure user agreement.

3.3.Dynamic Display of Registered Participants:

- **Data Extraction**: Programmed the script to capture values from textboxes, selected radio buttons, and the dropdown menu simultaneously upon submission.
- **DOM Table Updates**: Used the insertRow() and insertCell() methods to create a new row at the bottom of the existing table for every successful registration.

3.4 Prevent Page Reload:

- **Event Interception**: Utilized the e.preventDefault() method within the form's onsubmit function.
- **Seamless Experience**: This prevents the browser from refreshing or navigating away, allowing the participant table to update instantly while keeping all current session data intact.

3.5 Clear Form After Successful Submission:

- **State Reset**: Used the .reset() method on the form object to clear all input fields after the data has been safely moved to the table.
- **Validation Cleanup:** Added a loop to remove "redBorder" or "greenBorder" CSS classes from all inputs, returning the form to its original blank state.

3.6 Display Success Message:

- **Visual Confirmation:** Directed a "Registration Successful!" message to a dedicated statusMsg div with green text styling.
- **Auto-Removal**: Applied setTimeout() to automatically clear the success message after 3 seconds to keep the interface clean.

3.7 Input Field Feedback:

- **Visual Cues**: Programmed the script to dynamically assign CSS classes (redBorder for errors or greenBorder for valid data) based on validation results.
- **Instant Correction**: This provides the user with immediate visual confirmation of which fields need correction before they can proceed.

3.8 Optional Enhancements:

- **File Upload Validation**: Included a check to ensure an ID proof file has been selected before the form is considered complete.
- **Consistent Formatting**: Ensured that the dynamically added data in the table maintains the same professional alignment and styling as the header row.

4.JavaScript Concepts Used:

- **DOM Selection**: Used document.getElementById to access form inputs and the table.
- **Event Listeners**: Applied .addEventListener('submit', ...) to handle user actions.
- **Conditional Logic**: Used if-else statements to check for valid input formats.
- **Array/Object Handling**: Captured form values into variables before inserting them into the table.
- **Function Definitions:** Created reusable functions for validation and table updates.

5. Implementation Steps:

Script Link: Connected the external script.js to the HTML file.

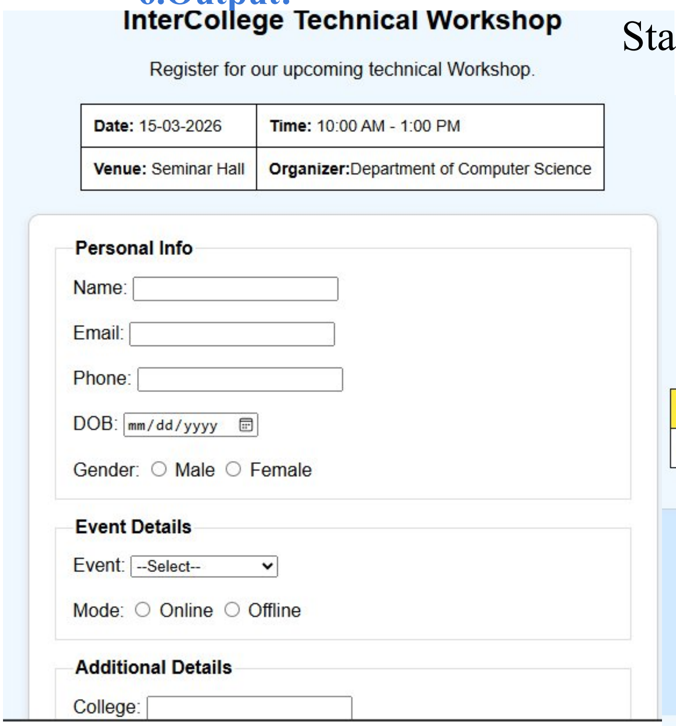
Validation Logic: Wrote functions to verify email patterns and phone number lengths.

Submission Handler: Prevented the form's default reload behavior to keep the dynamic data visible.

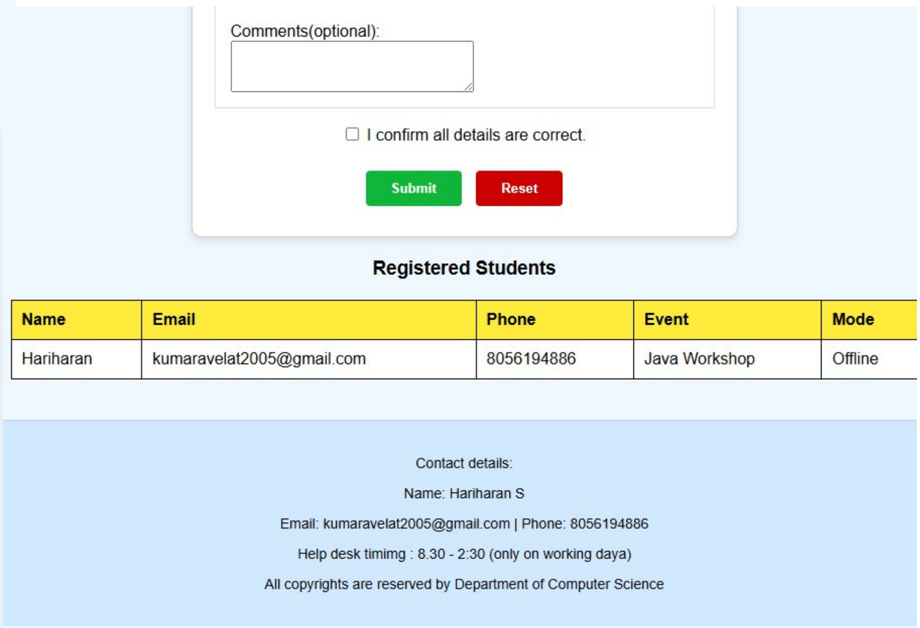
Table Insertion: Used insertRow() and insertCell() to add the new participant details to the bottom table.

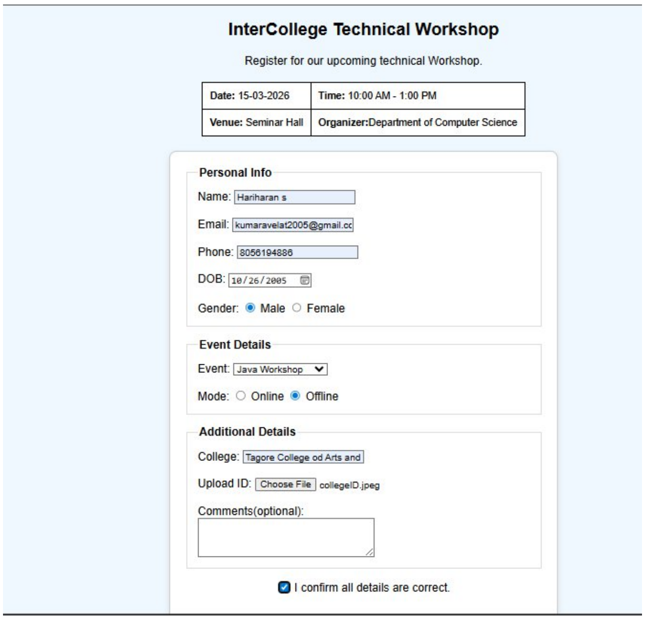
Cleanup: Triggered the reset() method to prepare the form for the next user

6.Output:



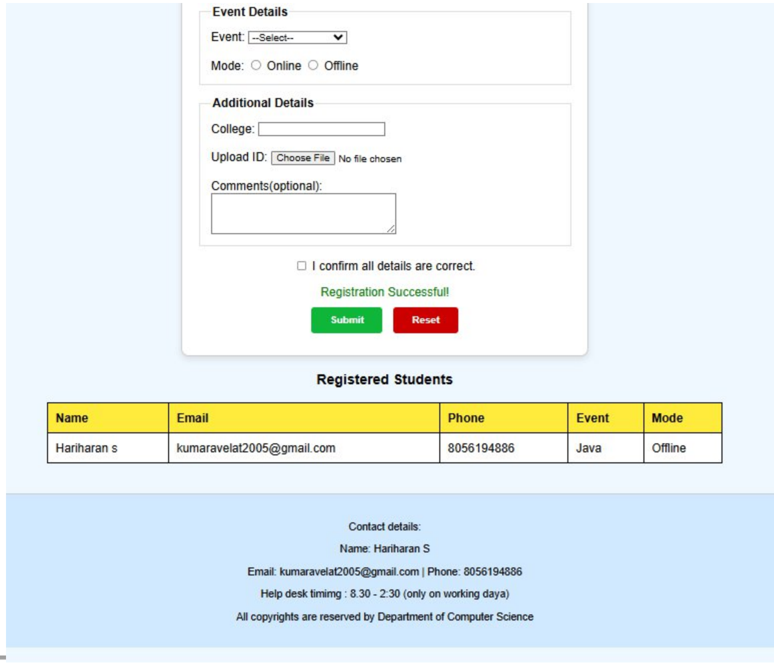
Static page





Before submit

After submit



7.Learning Outcome:

Interactivity: I learned how to move beyond static styling and create a functional web application using JavaScript.

Data Integrity: I understood how to enforce rules on user input to ensure the database (or table) only receives correctly formatted information.

DOM Skills: I gained confidence in manipulating the Document Object Model to change page content dynamically without a server.

8. Conclusion:

The project successfully integrates HTML, CSS, and JavaScript to form a complete registration system. The validation logic ensures data quality, while the dynamic table update provides immediate satisfaction to the user, proving that client-side scripting is essential for a modern web experience.